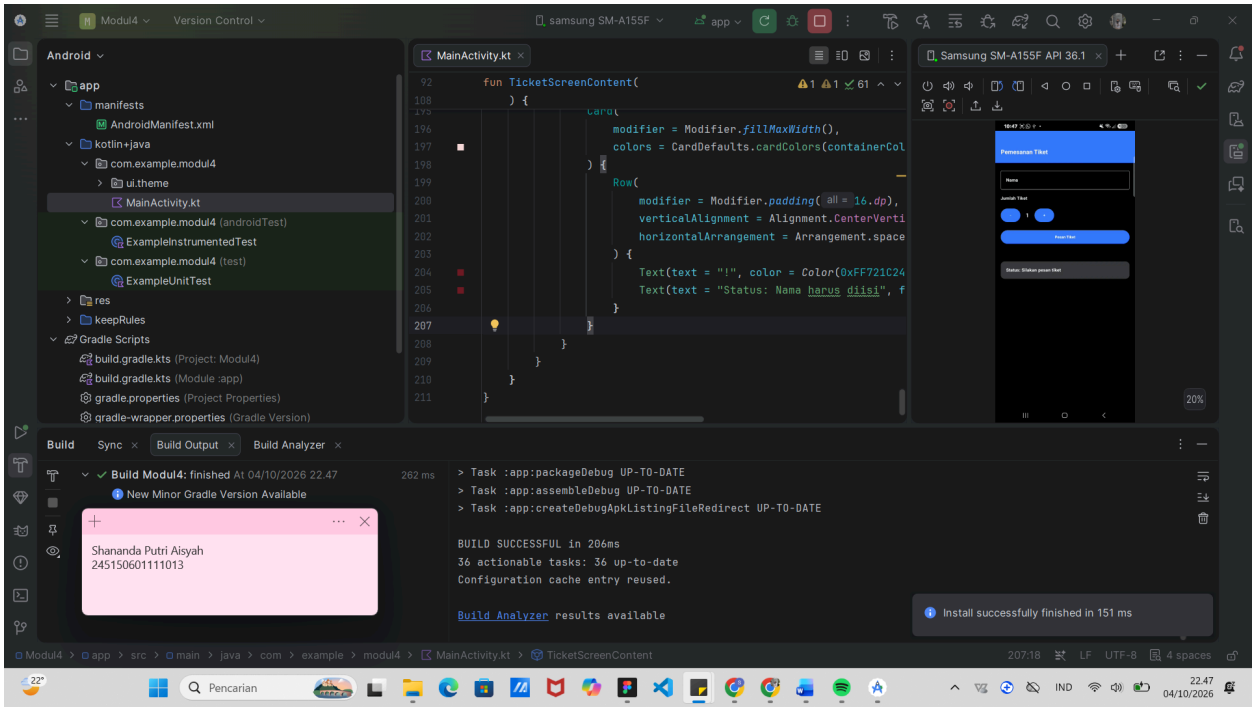
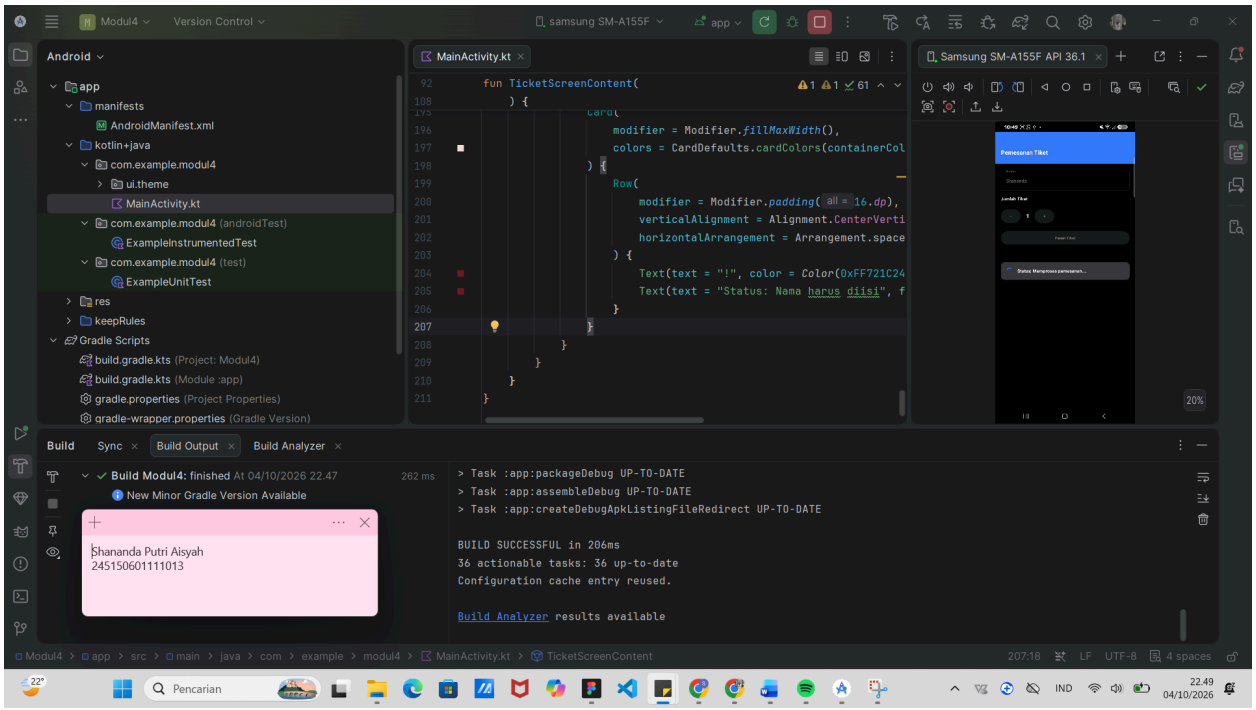
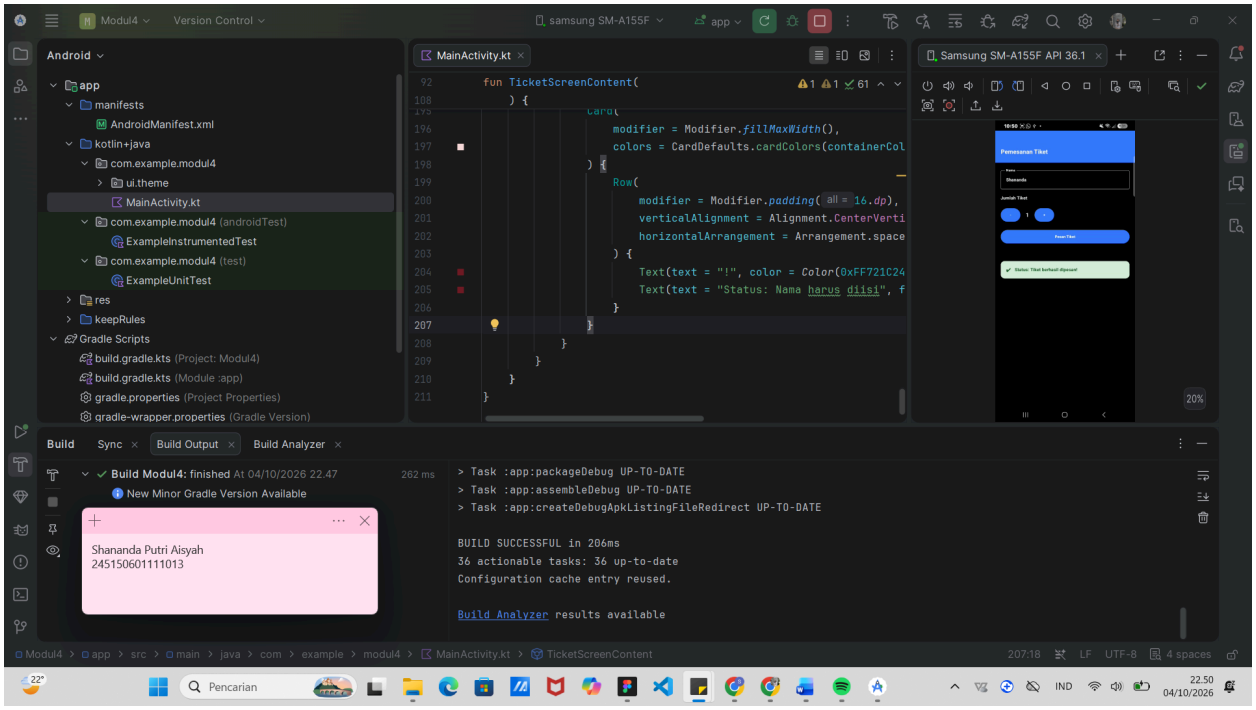
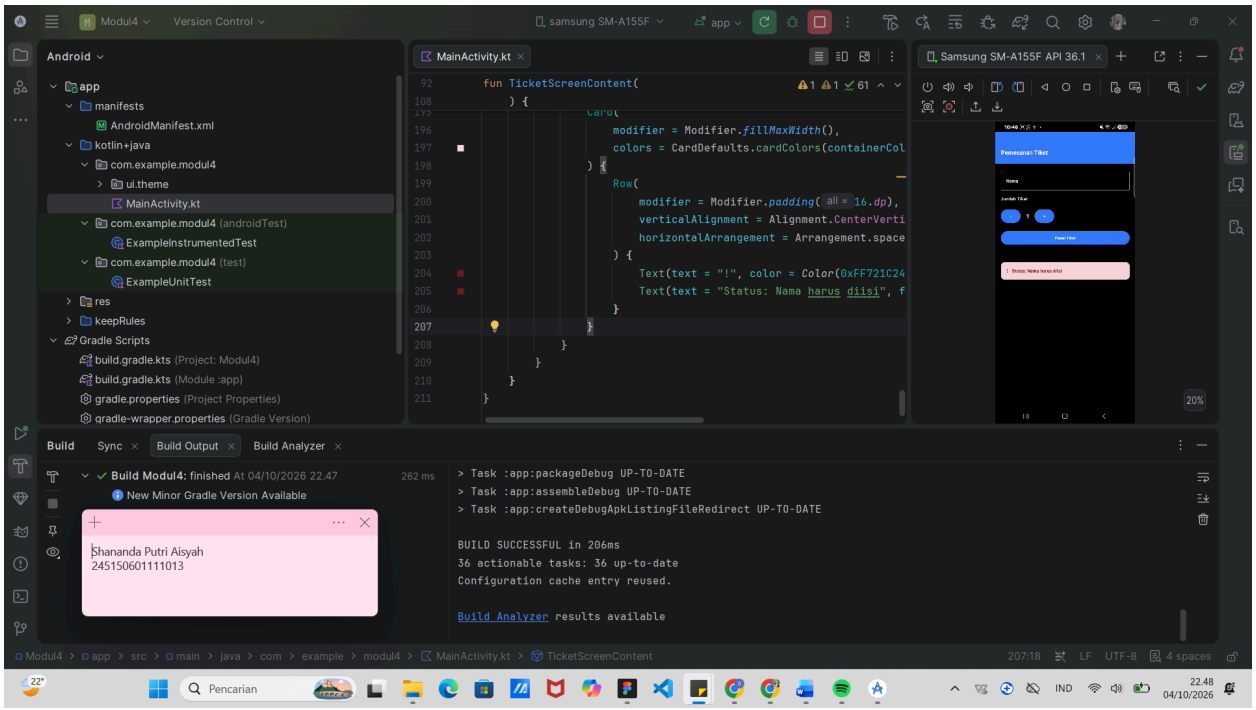
Nama : Shananda Putri Aisyah
NIM : 245150601111013
TUGAS PRAKTIK









Link Github : https://github.com/Shanandaputri/Modul4.git

Modul 4 : State Hoisting, Perubahan Konfigurasi, dan Side-effect

I. State Hoisting

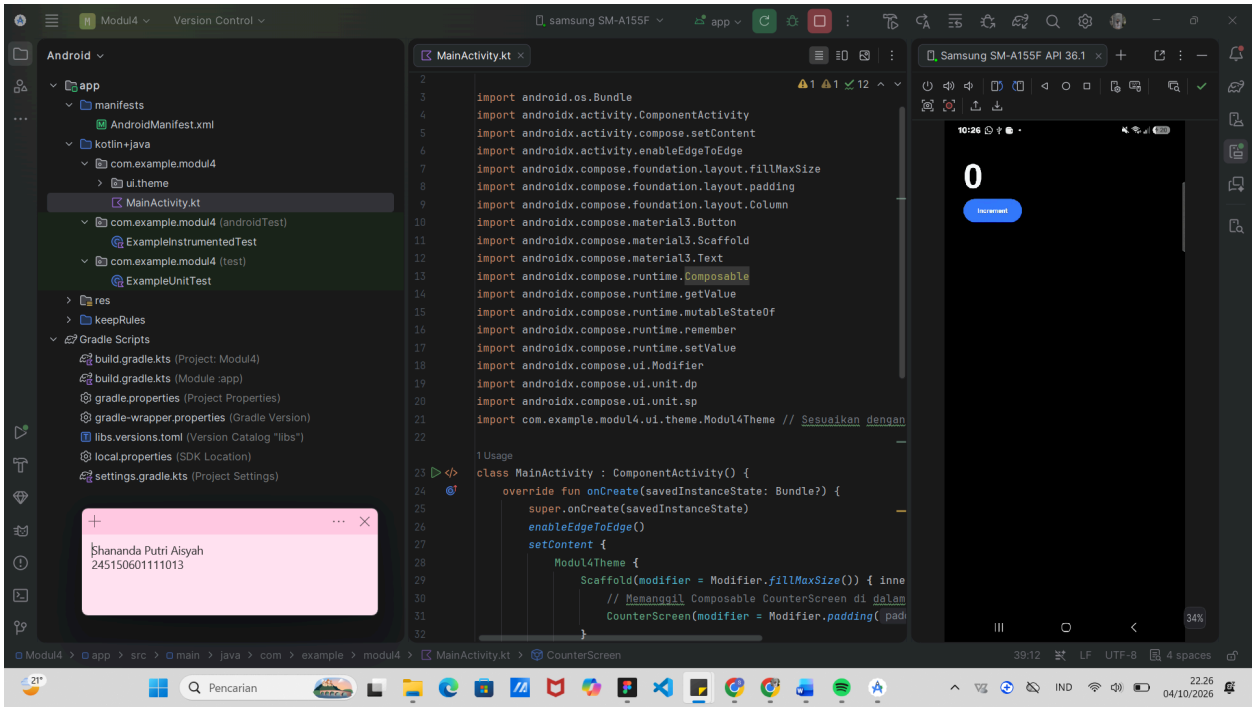
Aktivitas Percobaan

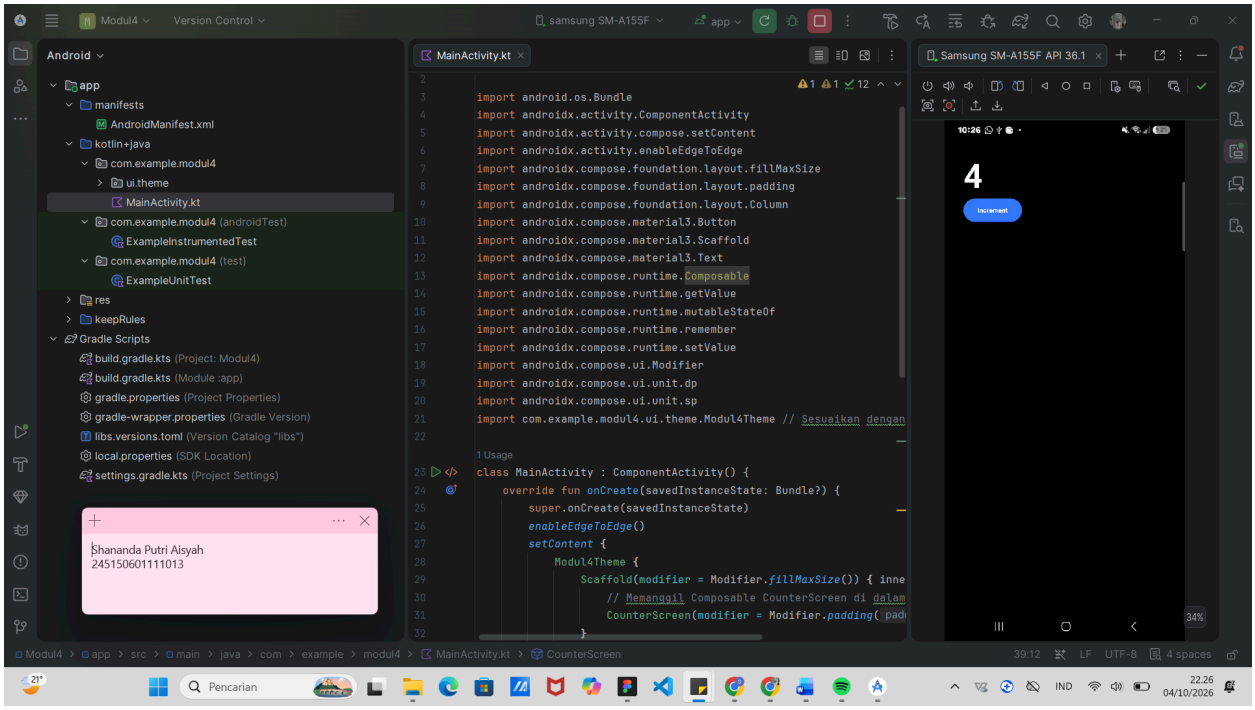
Nomor 1

A. Setiap kali tombol "Increment" diklik, nilai angka (number) yang ditampilkan pada komponen Text akan bertambah satu secara berurutan (misalnya dari 0 menjadi 1, 2, 3, dan seterusnya) melalui proses rekomposisi antarmuka.
B. Hal tersebut terjadi karena fungsi lambda onClick pada komponen Button memicu eksekusi perintah penambahan nilai (number++) pada variabel state, yang kemudian memberikan sinyal kepada Jetpack Compose untuk menggambar ulang (recomposition) komponen UI dengan nilai number terbaru.
C. Variabel state yang berperan adalah variabel number yang dideklarasikan menggunakan konstruk remember { mutableStateOf(0) }
D. Variabel state yang berperan adalah variabel label yang dideklarasikan menggunakan konstruk remember { mutableStateOf("Increment") }.

Nomor 2

A. Lokasi: Di dalam file MainActivity.kt, pada fungsi composable CounterScreen.
    a. Detail Perubahan:
        i. Mengubah logika penambahan nilai pada event klik tombol dari yang sebelumnya bertambah 1 (number++) menjadi bertambah 2 (number += 2).
        ii. Mengubah teks label pada tombol dari yang tadinya "Increment" menjadi "Increment 2" (atau melalui variabel state label yang nilainya diubah menjadi "Increment 2").

B. 

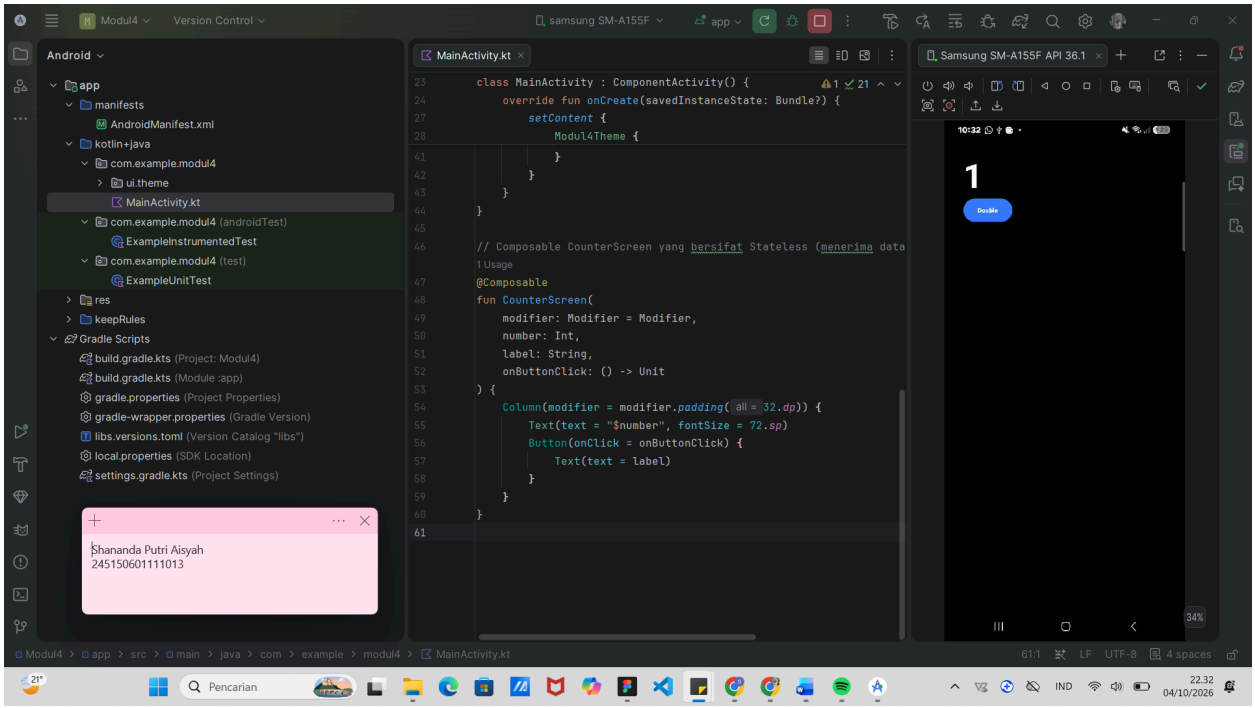


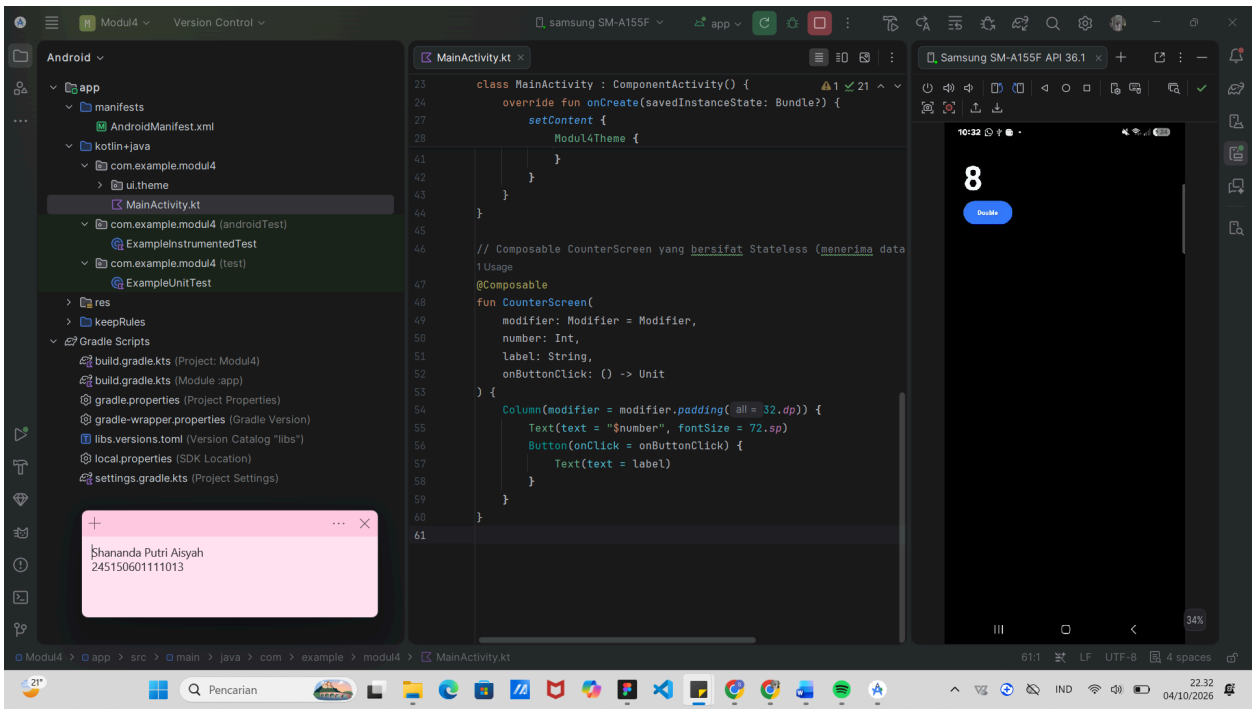
Nomor 4

A. Ya, program berjalan dengan baik serta menampilkan antarmuka dan fungsionalitas yang sama persis seperti pada tahap percobaan sebelumnya, tetapi kini strukturnya telah menerapkan pola arsitektur State Hoisting (komponen anak menjadi stateless).
B. Proses increment saat ini dilakukan di dalam fungsi induk (onCreate()), tepatnya di dalam fungsi lambda callback onButtonClick = { number++ } (atau sesuai logika increment yang diatur pada pemanggilnya).
C. State untuk komponen Text (number) saat ini dideklarasikan di dalam fungsi induk (onCreate()), sebelum fungsi composable CounterScreen dipanggil. Sedangkan untuk label tombol, nilainya dikirimkan secara langsung sebagai argumen parameter (label = "Increment") dari pemanggilnya (parent).

Nomor 6

A.





B. Perubahan dari tahap nomor 7 (penerapan state hoisting dasar) dan tahap nomor 8 (perubahan kebutuhan fungsional menjadi Double / penggandaan nilai) dilakukan di dalam fungsi onCreate() pada file MainActivity.kt, tepatnya pada bagian saat fungsi induk memanggil composable CounterScreen dan mendeklarasikan state utamanya.

C. Tidak ada perubahan kode program yang dilakukan di dalam fungsi composable CounterScreen itu sendiri antara tahap nomor 7 dan tahap nomor 8. Fungsi

CounterScreen tetap dibiarkan dalam kondisi stateless (hanya menerima parameter number, label, dan onButtonClick). Hal ini membuktikan manfaat dari State Hoisting, di mana perubahan fungsionalitas logika bisnis (seperti mengubah increment biasa menjadi perkalian/penggandaan double, mengubah label, atau mengubah nilai awal) cukup diselesaikan di tingkat pemanggil (parent/caller di dalam onCreate()) tanpa perlu menyentuh atau merubah struktur kode di dalam komponen anak (composable UI).

Nomor 7

Jawaban pada tahap nomor 9 (di mana perubahan logika bisnis seperti mengubah nilai awal, label tombol, dan operasi hitung dari increment menjadi double/penggandaan cukup dilakukan di fungsi induk/pemanggil tanpa menyentuh komponen anak) memiliki arti penting bahwa State Hoisting berhasil mewujudkan pemisahan kekhawatiran (Separation of Concerns) dan meningkatkan Reusability (kemudahan penggunaan kembali) pada antarmuka pengguna.

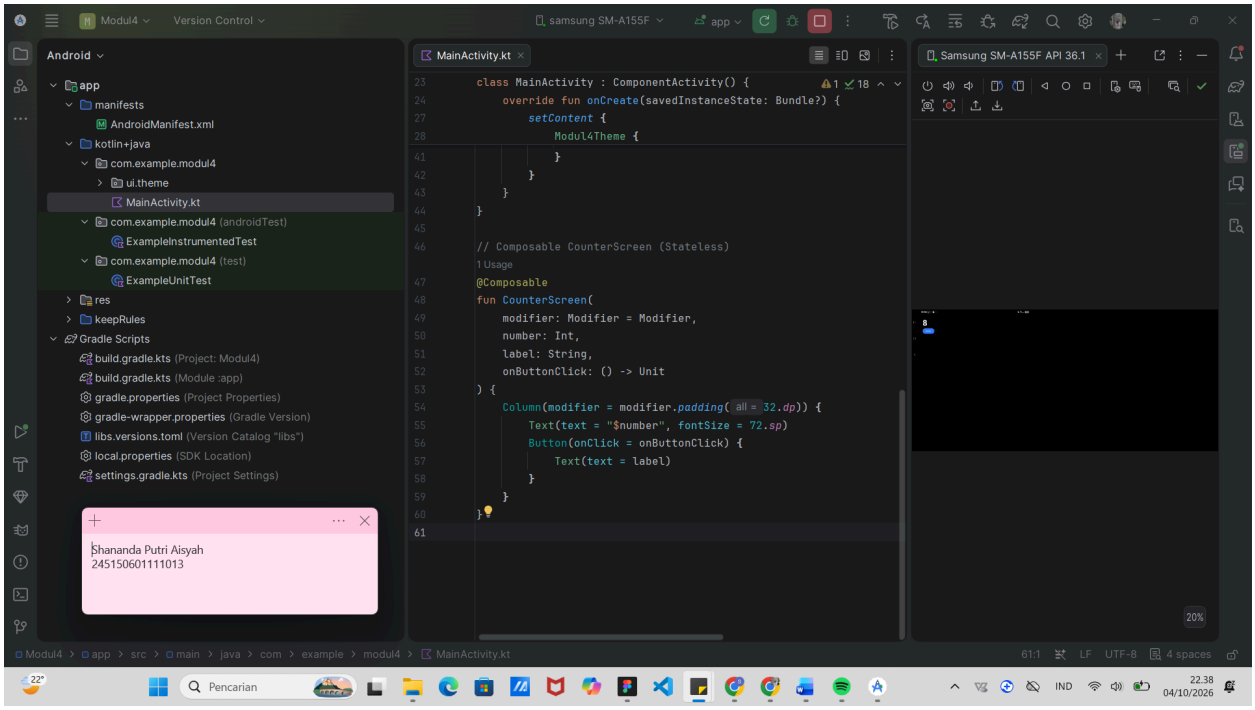
Manfaat utamanya terhadap perubahan kebutuhan fungsional aplikasi meliputi:

1. Komponen UI Bersifat Murni (Stateless): Komponen anak seperti CounterScreen hanya bertugas menampilkan data dan melempar event (callback), sehingga ia menjadi komponen yang fleksibel dan bersih dari logika bisnis.
2. Skalabilitas & Pemeliharaan Kode yang Mudah: Ketika terjadi perubahan kebutuhan fungsional dari sisi produk atau UI di kemudian hari, pengembang tidak perlu membongkar, merombak, atau berisiko merusak struktur komponen visual anak. Seluruh perubahan logika cukup diatur secara terpusat pada komponen induk (parent/caller).

TUGAS PRAKTIK

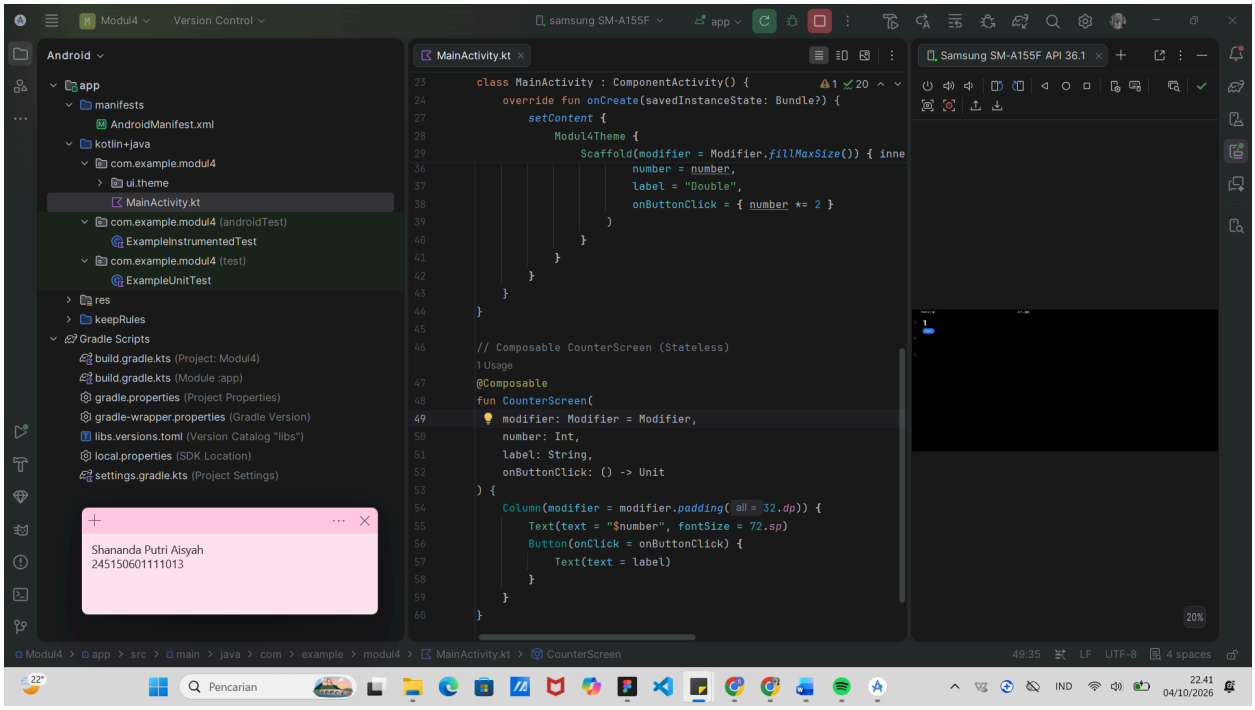
Perubahan Konfigurasi

Nomor 2

A. 

B. Angka yang ditampilkan: Angka terakhir yang diklik sebelum perangkat diputar tetap bertahan (tidak berubah atau tereset kembali ke nilai awal).
Mengapa terjadi demikian: Hal tersebut terjadi karena kode program telah diubah menggunakan konstruk rememberSaveable alih-alih remember biasa. rememberSaveable secara otomatis menyimpan nilai state ke dalam objek penyimpanan Bundle sistem Android, sehingga nilai state tersebut aman dan dapat dipulihkan kembali meskipun terjadi perubahan konfigurasi perangkat (seperti rotasi layar dari portrait ke landscape) yang menyebabkan Activity dihancurkan dan dibuat ulang (re-created).

Nomor 3

A. 

B. Angka yang ditampilkan: Nilai angka terakhir yang sama persis dengan kondisi sebelum perangkat diputar ke posisi landscape (tidak kembali ke nilai awal 1).
Mengapa terjadi demikian: Hal tersebut terjadi karena kode program telah menggunakan konstruk rememberSaveable, di mana nilai state secara otomatis disimpan ke dalam mekanisme sistem Bundle sehingga aman dan dapat dipulihkan kembali saat terjadi perubahan konfigurasi (seperti rotasi layar) yang menyebabkan Activity dihancurkan dan dibuat ulang (re-created).

C. Fungsi rememberSaveable: Berfungsi untuk mempertahankan nilai state tidak hanya selama proses rekomposisi UI berlangsung, tetapi juga ketika terjadi perubahan konfigurasi perangkat (seperti rotasi layar) atau pembuatan ulang Activity oleh sistem.
Perbedaannya dengan remember: Konstruk remember hanya menjaga state tetap ada selama rekomposisi UI berlangsung dan akan kehilangan nilainya (tereset ke nilai awal) jika Activity hancur akibat rotasi layar. Sebaliknya, rememberSaveable dapat menyelamatkan dan menyimpan data tersebut ke dalam objek Bundle sistem Android.

Nomor 4

Contoh Kasus Nyata:

1. **Formulir Pendaftaran / Pengisian Data (Sign-up / Checkout Form):** Kolom input teks yang panjang seperti nama lengkap, alamat email, nomor telepon, dan alamat pengiriman di aplikasi e-commerce atau pendaftaran akun.
2. **Status Filter atau Tab yang Dipilih:** Kondisi *checkbox*, tombol radio (*radio button*), atau tab aktif yang sedang dipilih oleh pengguna pada halaman pencarian atau pengaturan.
3. **Posisi Scroll pada Daftar Panjang (LazyColumn):** Posisi gulir (*scroll position*) saat pengguna sedang melihat daftar produk atau riwayat transaksi yang panjang.

Manfaat yang Diperoleh:

- **Mencegah Hilangnya Data Pengguna (*Data Loss Prevention*):** Pengguna tidak perlu mengetik ulang atau mengulangi pengisian formulir dari awal apabila tanpa sengaja atau sengaja memutar posisi perangkat mereka (*rotasi layar*).
- **Meningkatkan Pengalaman Pengguna (*User Experience / UX*):** Aplikasi terasa jauh lebih responsif, stabil, dan ramah pengguna karena state/kondisi aplikasi tetap utuh di tengah perubahan konfigurasi sistem.

LaunchedEffect

Nomor 1

1. Pada layar ditampilkan antarmuka aplikasi penghitung dengan nilai angka awal, dan sebuah komponen SnackBar muncul di bagian bawah layar yang menampilkan pesan berupa teks: "Number 1 is shown!".
2. Pesan tersebut ditampilkan karena fungsi LaunchedEffect(number) dirancang untuk langsung dieksekusi secara otomatis oleh sistem pada saat komponen Composable pertama kali masuk ke dalam komposisi (initial composition).
3. Pemicu: Pemicunya adalah evaluasi awal dari parameter key pada LaunchedEffect yang mendeteksi nilai variabel number (yang bernilai awal 1) saat pertama kali Composable dimuat.

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}
```

Nomor 2

1. Nilai angka pada tampilan layar berubah (berlipat ganda), dan secara bersamaan komponen SnackBar langsung muncul kembali atau memperbarui pesannya untuk menampilkan informasi angka terbaru (misalnya menampilkan teks "Number 2 is shown!" atau sesuai hasil perkalian angka saat itu).
2. Hal tersebut terjadi karena fungsi LaunchedEffect(number) mendefinisikan variabel number sebagai parameter kunci (key). Ketika tombol "Double" diklik, nilai variabel number mengalami perubahan, sehingga LaunchedEffect mendeteksi adanya perubahan pada key-nya.
   Sesuai cara kerjanya, ketika key pada LaunchedEffect berubah, ia akan otomatis membatalkan proses/efek sebelumnya dan menjalankan ulang blok kode di dalamnya untuk menampilkan SnackBar dengan nilai number yang baru.

Aktivitas Percobaan

Nomor 1

1. Pada saat pertama kali program dijalankan, aplikasi menampilkan nilai angka awal (misalnya angka 1) dan SnackBar menampilkan pesan default sesuai nilai awal tersebut.
2. Setelah menunggu selama dua detik, nilai angka pada layar berubah secara otomatis menjadi angka acak (random) yang dikembalikan oleh fungsi pengambilan data dari repositori (Repo.getData()), yaitu berkisar antara 100 sampai 1000.
3. Pemicu: Perubahan tersebut dipicu oleh blok LaunchedEffect(Unit) yang menjalankan fungsi asynchronous (simulasi jaringan dengan delay(2000)), lalu memperbarui nilai variabel number dengan data baru dari Repo.getData(). Perubahan pada nilai number ini kemudian mentrigger LaunchedEffect(number) untuk menampilkan kembali SnackBar dengan nilai yang baru.

```
LaunchedEffect(Unit) {
   number = Repo.getData()
}
```

Nomor 2

1. LaunchedEffect(Unit): Menggunakan parameter kunci (key) berupa Unit (objek konstan yang tidak pernah berubah). Karena nilainya konstan, blok kode di dalamnya bersifat statis dan tidak akan pernah di-restart ulang akibat perubahan state.
   LaunchedEffect(state): Menggunakan parameter kunci berupa variabel state tertentu yang dapat berubah nilainya. Jika nilai pada parameter state tersebut berubah, LaunchedEffect akan otomatis membatalkan (cancel) coroutine yang sedang berjalan sebelumnya dan menjalankan ulang (re-launch) blok kode di dalamnya.
2. **Operasi dalam LaunchedEffect(Unit) dijalankan:** Hanya **satu kali** saja, yaitu tepat pada saat komponen *Composable* pertama kali masuk ke dalam komposisi (*initial composition* / saat pertama kali layar dimuat). Sangat cocok untuk proses inisialisasi awal seperti *fetching* data dari server/database.

Operasi dalam LaunchedEffect(state) dijalankan:

1. Pertama kali saat *Composable* masuk ke dalam komposisi.
2. **Setiap kali** terjadi perubahan nilai pada variabel *state* yang dijadikan sebagai parameternya.